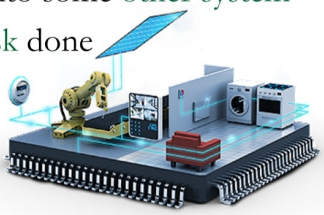
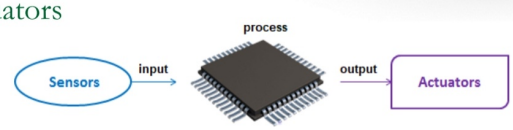
Introduction to Embedded Systems

25 / 06 / 2026

Embedded System

- Computer system embedded into some other system
- Used to get some dedicated task done
- ~~Preforms~~ Physical Computing → Perform
  - Sensors
  - Processor
  - Actuators





- Usually operates under strict constraints
  - What are they?

*Sensors capture state of the actual world, which you feed to the processor as an input.

*Processor processes the inputs. (→ prod outputs) according to instructions provided. (By programming)

↓

Known as Firmware (It doesn't change)

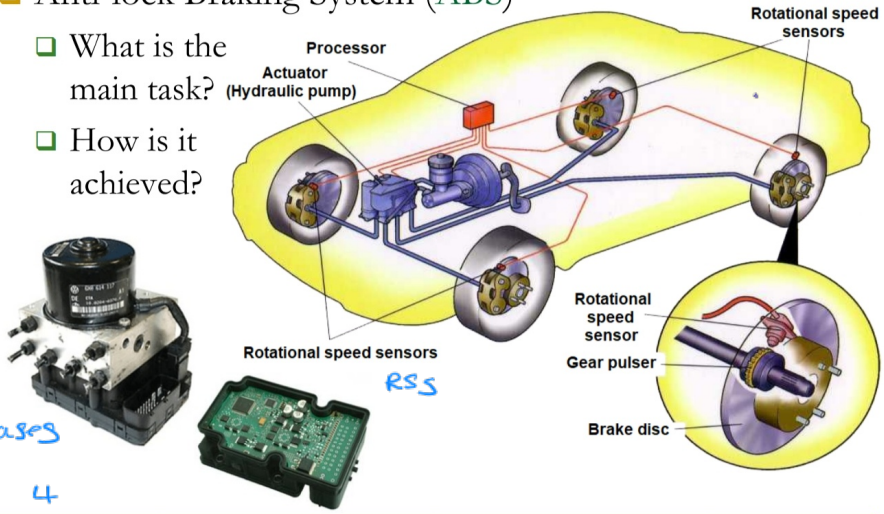
*The processed outputs drive the actuators (ex: Motor of a fan)

ex: Sense temp, if >30, turn on Fan.

Constraints : 1. Cost (ES Must be cheap)
2. Weight/Size (Must be small)
3. Power Consumption (Must consume least amt of Power)
4. Real-time Performance

Example

- Anti-lock Braking System (ABS)
  - What is the main task?
  - How is it achieved?



* Main task → Apply equal break to all 4 wheels

*How? RSS measures Rotational speed per min.

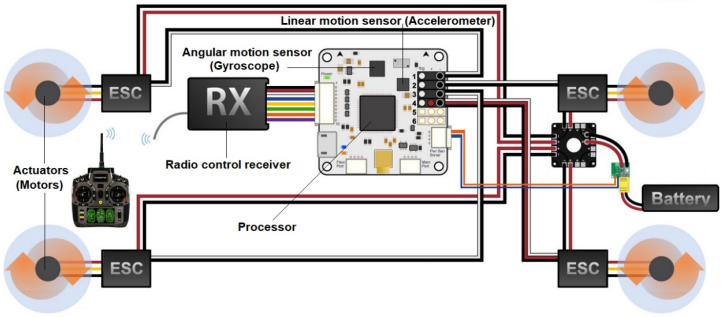
Actuator → Hydraulic pump

If one wheel has less brakes than other 3, the pump releases fluid to that wheel until all 4 wheels are rotating at the same speed.

Example

- Flight Controller System
  - What is the main task?
  - How is it achieved?





Main Task:

* Stably Fly the drone.

How:

Sensors - Gryoscope

- Accelerometor

- Radio control receiver (receive pilots radio signals)

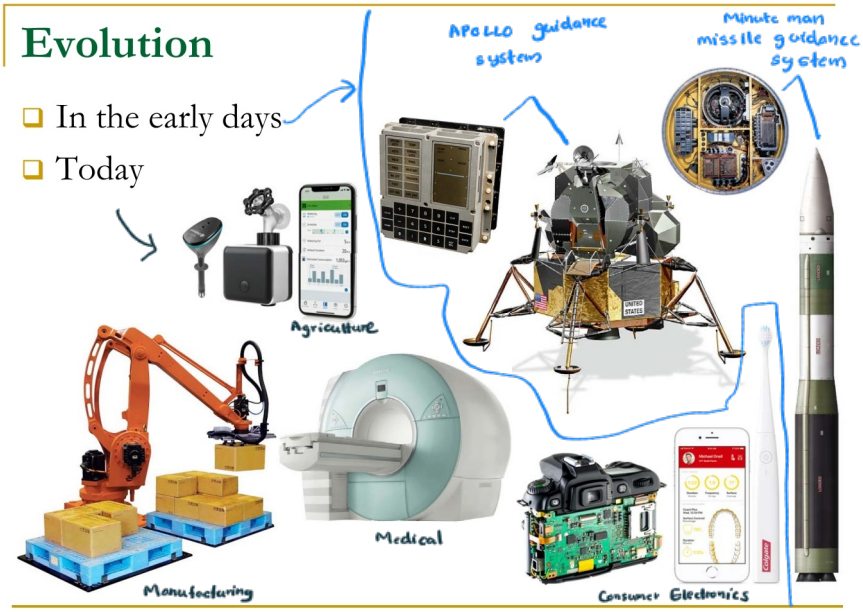
* Processor varies the speed of the 4 motors (actuators) based on the inputs.

* Early Days - Military

? Scientific purposes only cuz expensive

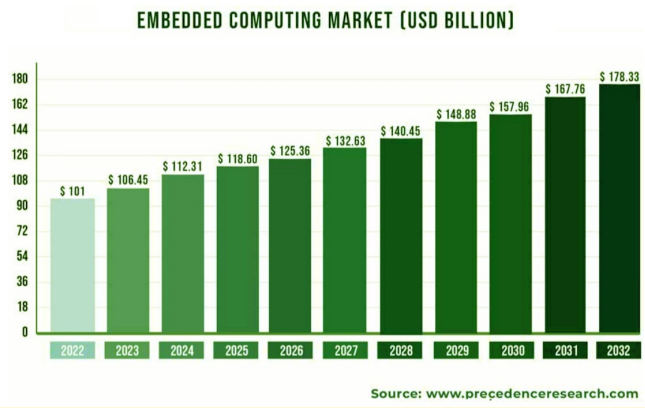
Evolution

- In the early days
- Today



Future

- Why are we interested in embedded systems?



Shift of Industry

- What's the secret behind exponential revenue growth?



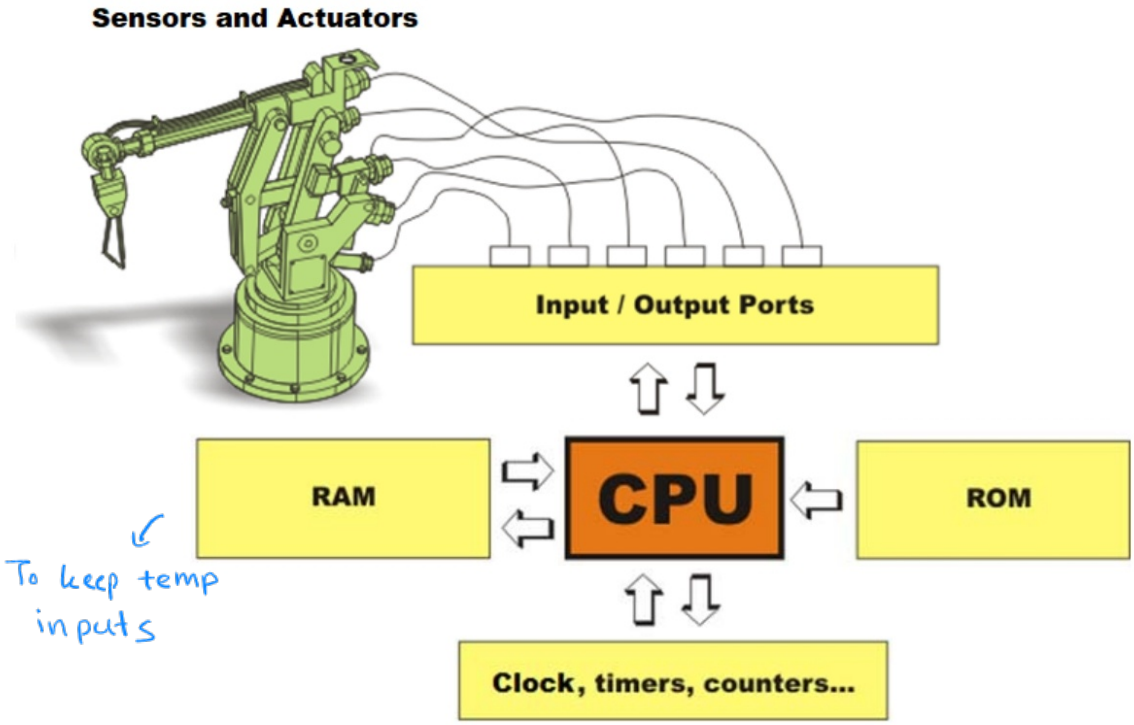
Majority of functionalities have shifted from hard wired electronics to software.

* Software implemented Functions aka Embedded Systems.

- What are the advantages?

1. Easier to implement. (Easy to get software on top of generic hardware)
2. Low Cost
3. Easier to debug
4. Easier to modify.

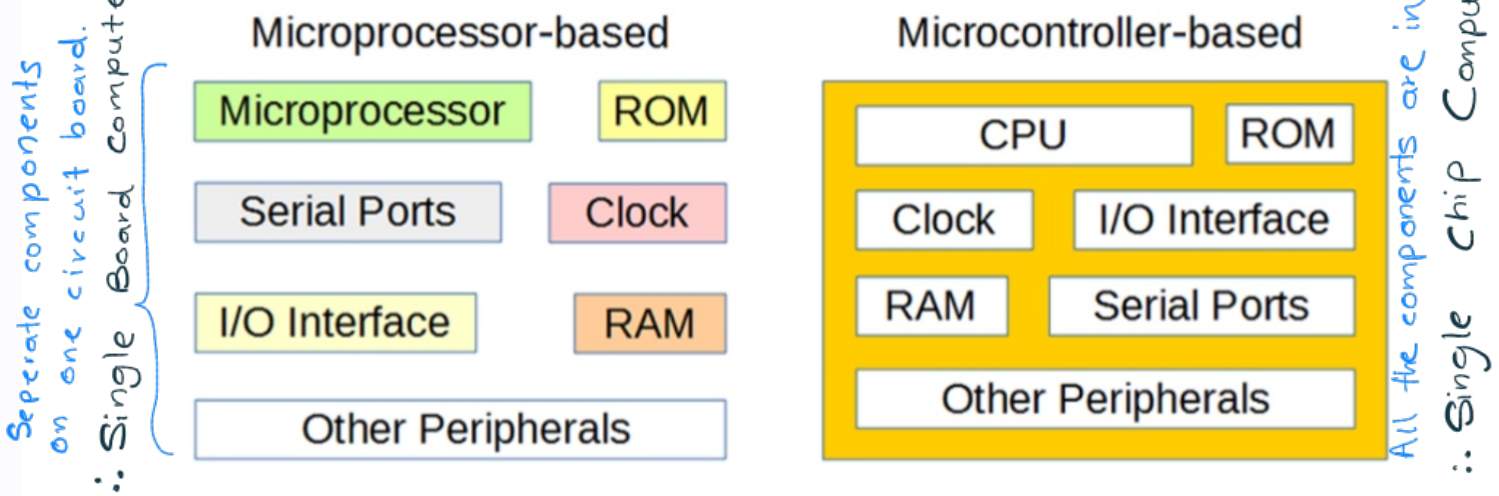
Components



To keep temp inputs

Implementations

- Microprocessor-based vs. Microcontroller-based



Seperate components on one circuit board.
∴ Single Board computer

All the components are in 1 chip
∴ Single chip Computer.

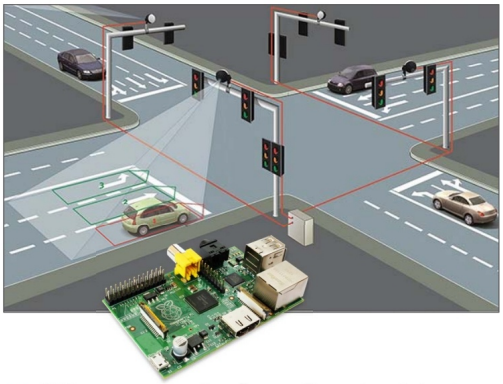
- What are the pros and cons?

* Spec is very high * Higher performance cuz components are seperate * Can replace each component	* Cannot replace seperate components * Everything is miniature ∴ not high spec
* High spec ∴ components are expensive, * Consume lot of power & generate lot of heat	* Cheap * Power efficient & less heat * Compact

Implementations

- Microprocessor-based or Microcontroller-based?

→ Depends on the system.

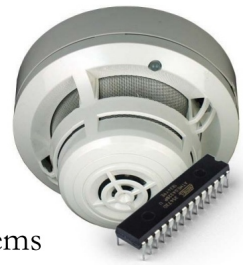


→ ex: Adaptive traffic control system.

→ Data coming from camera requires image processing. For that you need lot of RAM.

∴ this will have to be Microprocessor based.

Also no space or power supply constraints.



- Why are majority of embedded systems microcontroller-based?

For most ES power hungry, bulky systems are not needed.

→ ex 2: Fire Alarm system.

* Here Microcontroller based

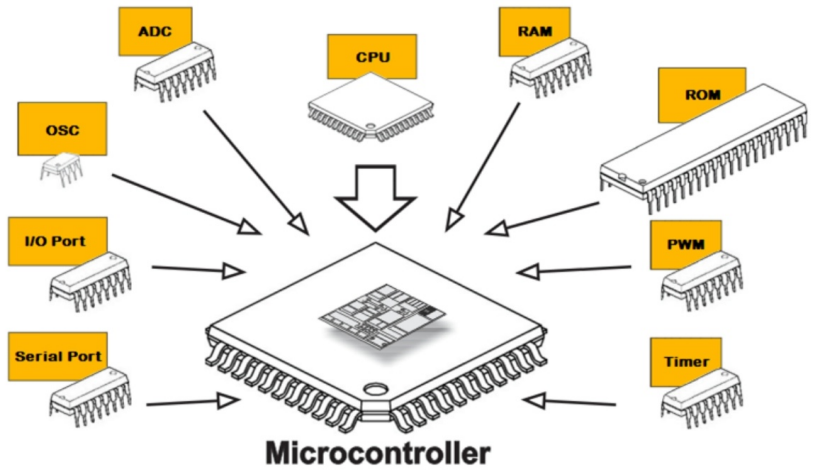
Because binary sensors are used inputs & outputs are binary values. ∴ large processing power is not required.

* Also operated by a battery & is small. ∴ low power consumption & compactness is a necessity. Must be low cost as well cuz fire alarms are bought in bulk.

02. Introduction to Microcontroller (MC) 02 / 07 / 2026

Microcontroller

- What is a microcontroller?



Microcontroller is a single chip computer.

*ROM is larger than RAM bcs :

RAM is used to store variables & controllers which are limited while ROM is used to store Firmware which is bulky.

*ADC : Analog to Digital Converter used to convert signals as processer can only understand digital signals. (Most inputs are analog → velocity, temp, speed etc)

* OSC : Oscillator is used to give precise clock pulses (1, 0, 1, 0 ...) to the processor.

* PWM : Used to give Analog like output (Pulse with Modulator) (ex: Not direct ON/OFF but dim a light)

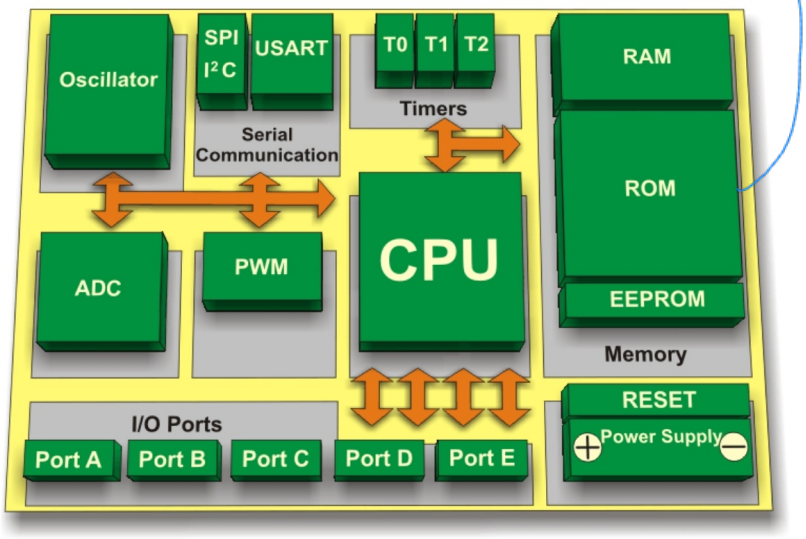
* Timer : To measure elapsed time.

*I/O ports : To get inputs and deliver outputs

* Serial port : For data communication. (ex: MC - MC communication)

Aka Flash bcs its writable but only by the developer.

Core Components



* EEPROM : To keep the settings of the system.
(Things that are non-volatile but changeable)

* Developer stores firmware in ROM while EEPROM data is stored by user.

* Serial Communication. (Data Communication)
  - → USART - Universal Synchronus Asynchronus Receiver Transmitter. (MC - PC comm)
  - → SPI - Serial Peripheral Interface } (MC - MC / memory chip)
  - → I²C - Inter-Integrated Circuit } nearby devices

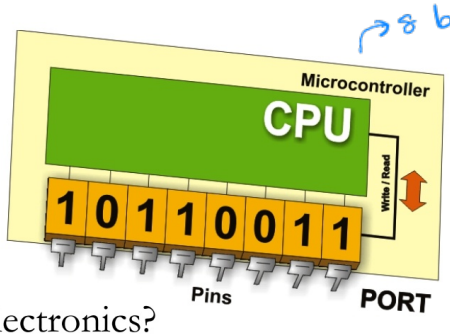
* 2 Methods for nearby comm bcs :
  - SPI is fast but use 4 wires
  - I²C is slower but uses only 2 wires (aka Two Wire Interface (TWI))

* I/O Port categories (A, B, C ...)
  - More ports mean more i/o pins. The no is decided depending on the requirements

* RESET - Used to reset the MC. Memory cleared & instructions are executed from the very first instruction.

I/O Port

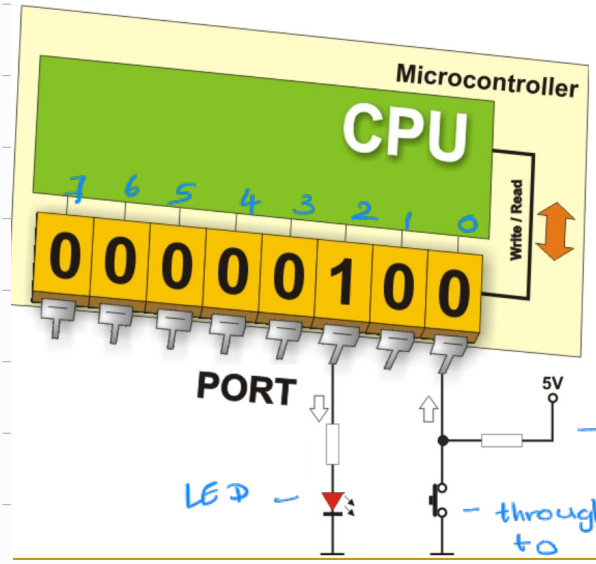
- What's an I/O Port?
- How can external electronics alter the flow of program execution?
- How can program in execution drive external electronics?



↗ 8 bit register

* I/O port is a register associated w MC pins.

Internal Working




```
Pseudocode
if bit 0 of PORT = 0
    bit 2 of PORT ← 1
else
    bit 2 of PORT ← 0
end if
```

Pin 0 is connected:
- through the resistor to 5V
- through the switch to ground

* Pins we count like arrays (0-7)

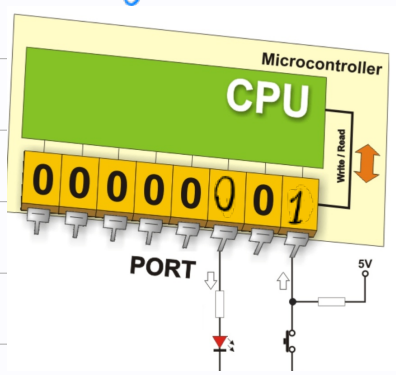
- Logic high : 2-5V
- Logic low : 0-1V

* Bit value represents Logic level of the pin.

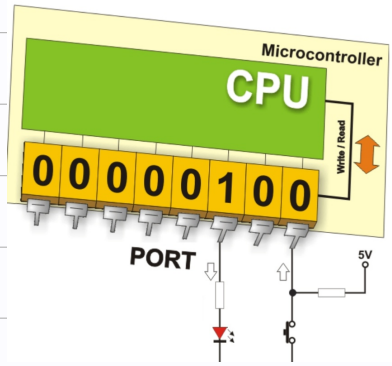
(1) * When switch is off pin 0 connected to 5V, ∴ logic level high shows bit value 1.

(2) * When switch is on (in image) pin 0 connected to ground directly ↓ (No resistance)
∴ logic level low shows bit value 0.

(1)

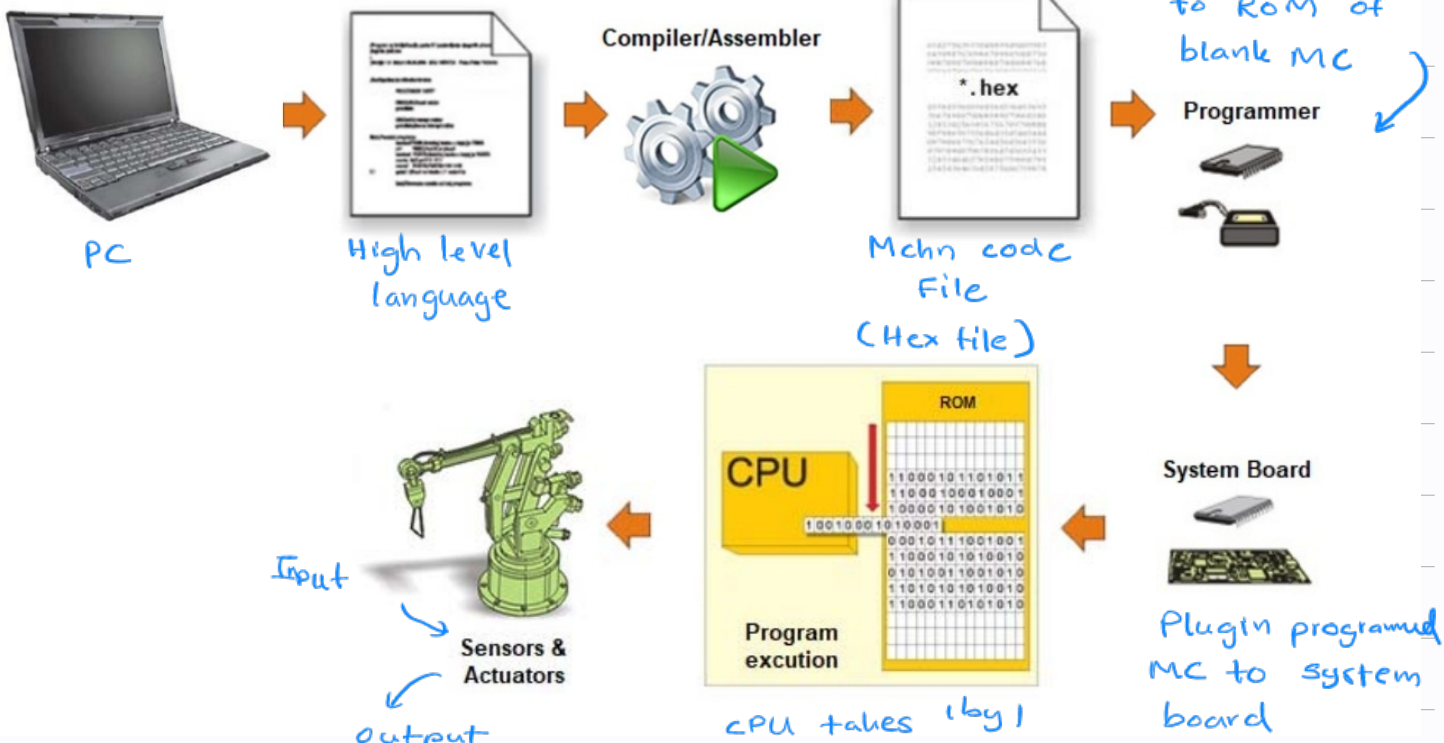


(2)



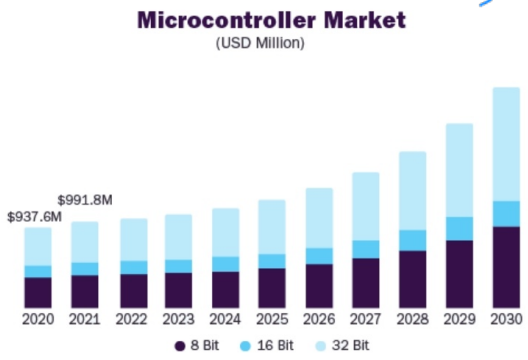
Extra: resistor is used to prevent short circuit from 5V being direct connected to ground

Programming



Word Size (No of bits processed at a time)

- 8, 16 or 32-bit?
  - Trend (8 has no growth)





→ Compound Annual Growth Rate.

  - Ease of learning 8 is easier to learn considering 32
  - Cost 8 is also cheaper (+ free & open src tools) (+ large community)

* When selecting MC need to consider
  ↳ Academic purposes
  ↓
  Ideal selection & bit.
  Later can migrate to 32

Families

- 8-bit families
  - 68HC08/68HC11 from NXP (formerly Motorola)
  - 8051 from Intel
  - AVR from Microchip (formerly Atmel)
  - PIC from Microchip
  - STM8 from STMicroelectronics
- Why AVR?

Atmel → No

ATmega328P

series



- Features
  - Harvard RISC Architecture
  - 131 single clock cycle instructions (mostly)
  - 8-bit 20 MHz CPU with 14-bit PC → Recommended but overclock possible.
  - 32 KB 16-bit word addressable program ROM
  - 2 KB byte addressable RAM and 1 KB EEPROM
  - Three I/O ports with 23 I/O pins
  - 3 Timers, 6 PWM channels and 6-channel 10-bit ADC
  - One USART, two SPI and an I^2C
  - 28 pin PDIP – Plastic Dual Inline Package. (Imp for academics as it is breadboard compatible)
- Why ATmega328P?

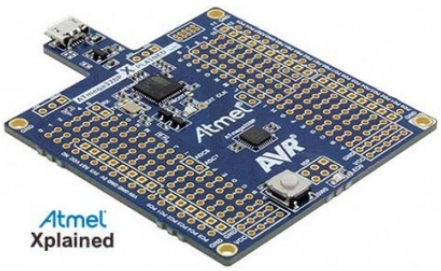
↳ Bcs it is the most popular MC

Most documented MC

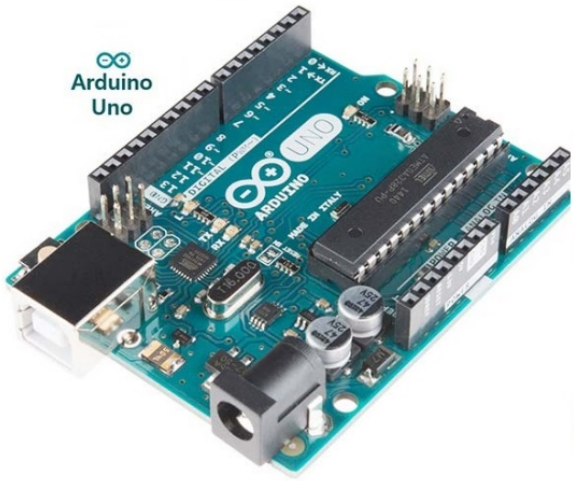
Heart of Arduno Uno Development Board – Mostly used dev board.

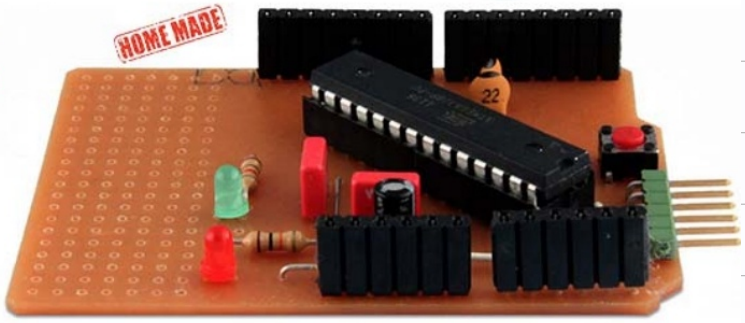
Development Boards

- Microcontroller-based









Arduino

- An open-source, low cost, easy-to-use hardware and software platform for prototyping embedded systems


Uno


Mega


101


Zero


Yun


Lilypad


Nano


MKR Zero

- For more information, visit https://www.arduino.cc/

Arduino Uno

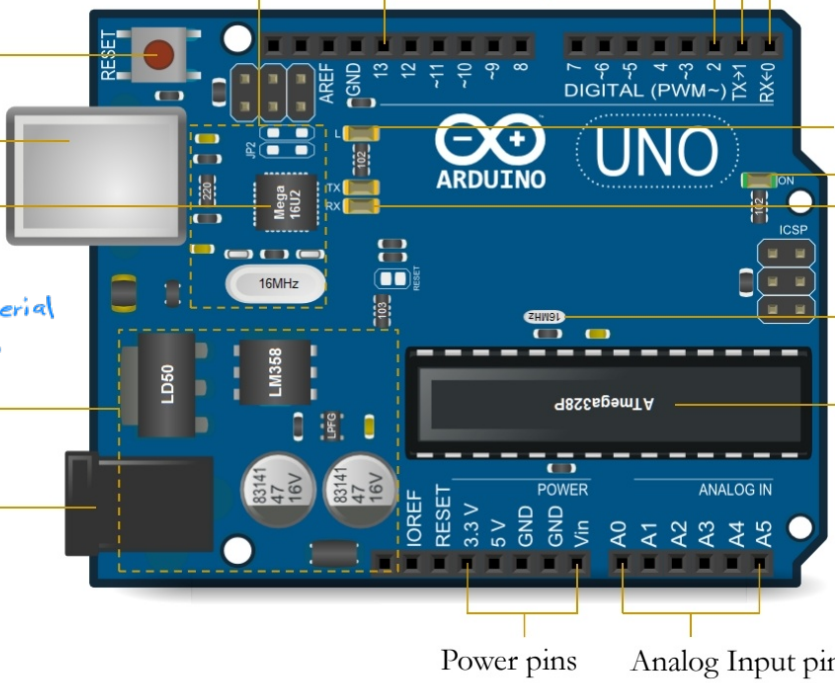
~ symbol : PWM supported pins

Transmit

Receive

3 data

Features



data comm

* Tx & Rx used to communicate w another board.

B1 : Tx Rx

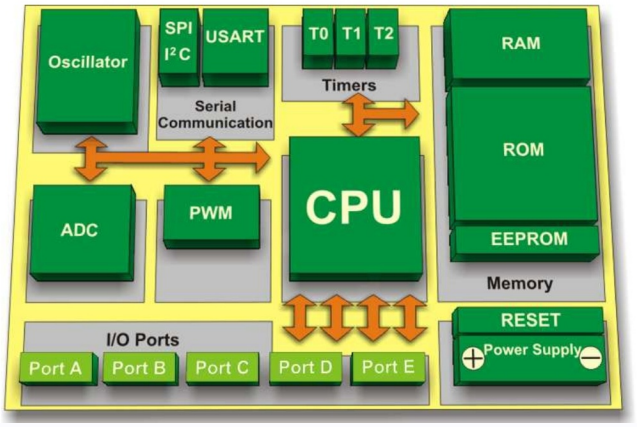
B2 : Tx Rx

* Sensors / Actuators are not connected to Tx/Rx pins bcs it will interfere w data communication.

03. I/O Port Programming.

I/O Ports

- Used to get the state of physical world via sensors
- Also used to drive actuators and thereby change the state of the physical world



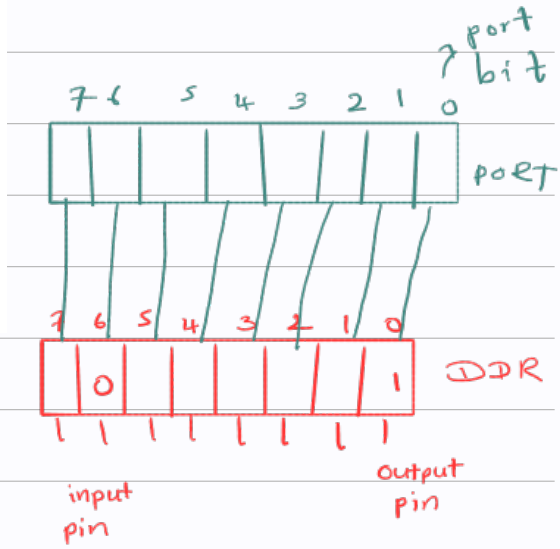
- Inputs to connect sensors
- Outputs to connect actuators

I/O Registers

- Registers associated with an I/O Port
- Data Direction Register (DDR)
  - Determines data direction
  - Set to configure port pins as outputs
  - Cleared to configure port pins as inputs
- Port Output RegisTer (PORT)
  - Used to specify the state of port pins

logic

→ Data in - input
out - output
to configure this DDR is used.



*If PORT register becomes 1 - high
0 - low

DDR and PORT Registers

DDR

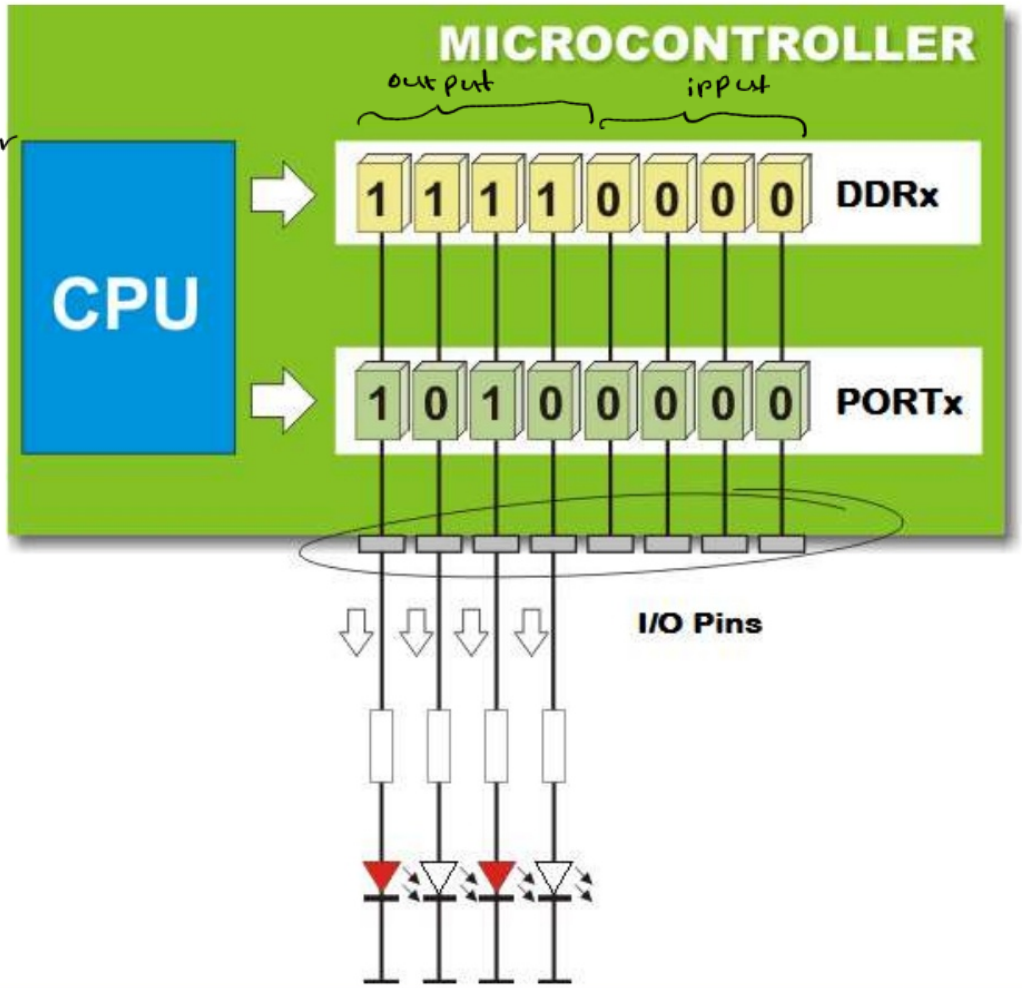
0 - input - clear

1 - output - set

PORT

1 - high - LED on

0 - low - LED off



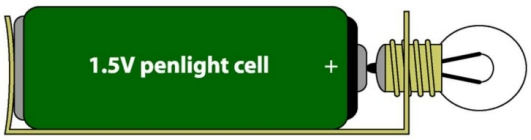
* x bcs there can be many (a, b, c, ... x)

• System 01

* blinker Asm

Electricity

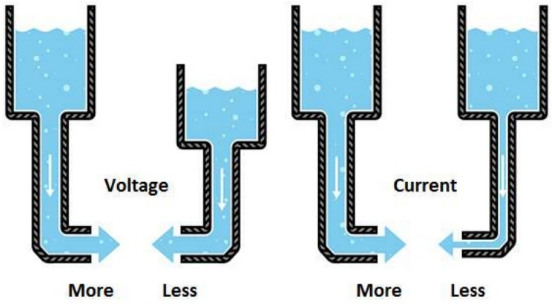
- Current
- Voltage



Same flow

current flow
pressure ↑

voltage ↑

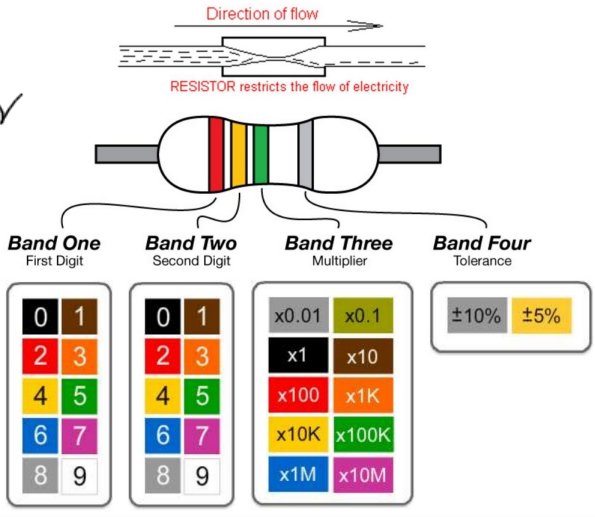


electrons ↑

current flow ↑

Resistor

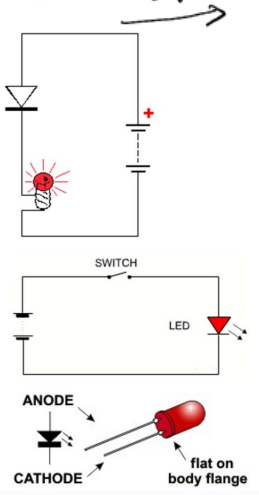
2.4MΩ

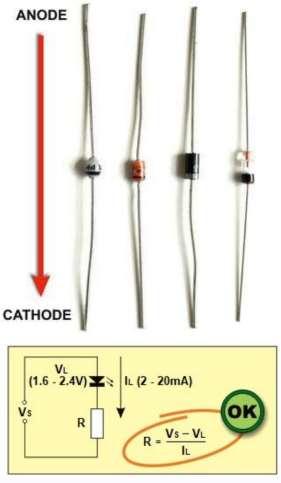


Diode

anode cathode

colour band indicate cathode

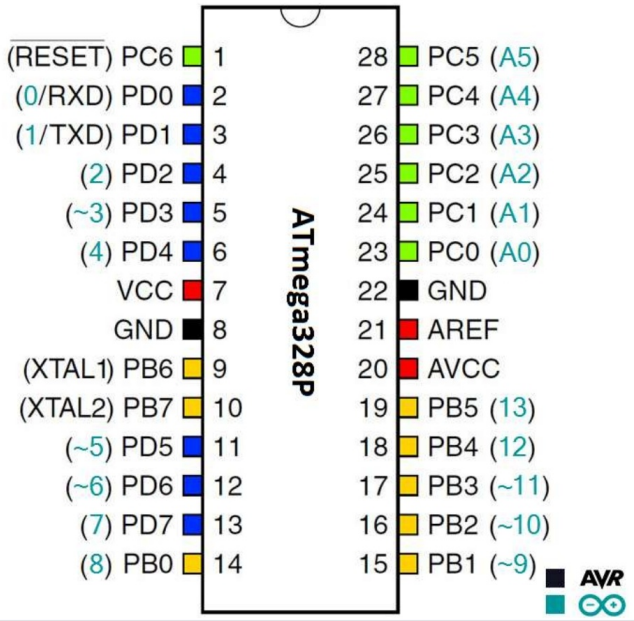




V = IR

/ /

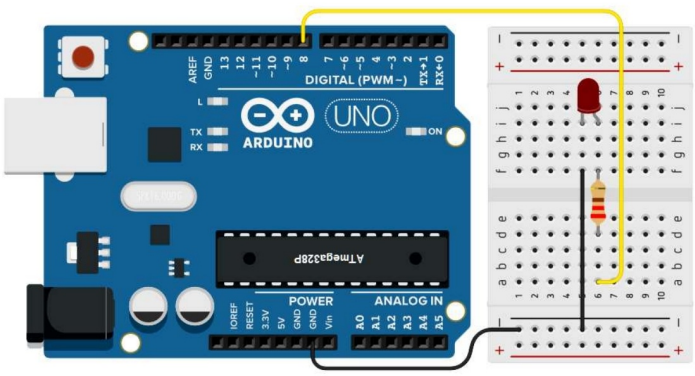
ATmega328P I/O Ports

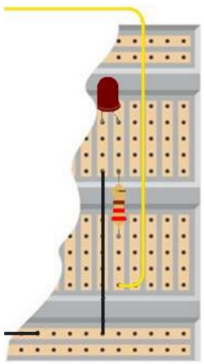


*This diagram will be given for the exam.

Hardware

- Wiring Diagram





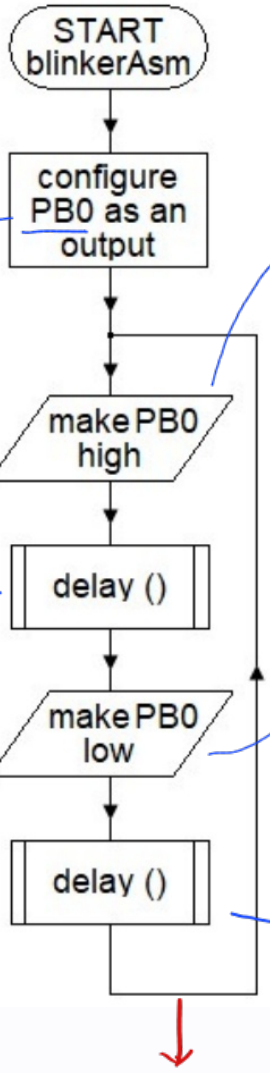
Vertically connected

Horizontally connected

* Switching on and off an LED with ½ a second time intervals.

Firmware

Algorithm



LED connected to this pin

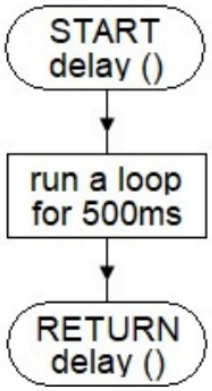
* That pin is I/O. For this we are using for output. ∴ configure as output

Logic high
∴ 5V
→ pin voltage 5V
GND " 0V
} Voltage diff ∴ current flows

Subroutine symbol
(delay is a func^{n})
↓
½ a second LED on

Logic Low
∴ 0V

½ second LED off



To explain the subroutine we need another FC

↳ without doing anything consume clock cycles for 500ms
↓
Purpose of the loop

Since there is no OS, we can only do line by line execution.

* Since there is no control (control fall out) we run the code again to prevent it.

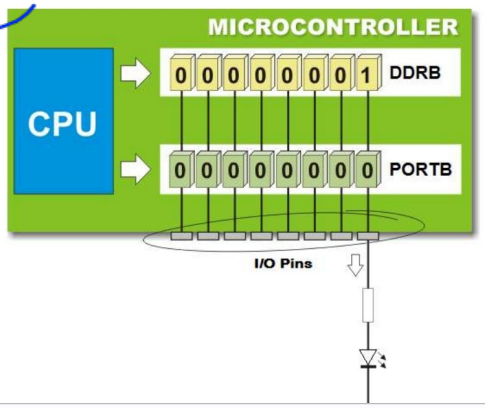
∴ having an endless loop is a must.

Configuring an I/O Pin

- Configure Pin 0 of Port B as an output
  - Use Set Bit in I/o register instruction

```
SBI DDRB, 0 ; set bit 0 of DDRB & configure PB0 as an output
```

Start of comment



SBI - Set Bit I

SBI DDRB, 0

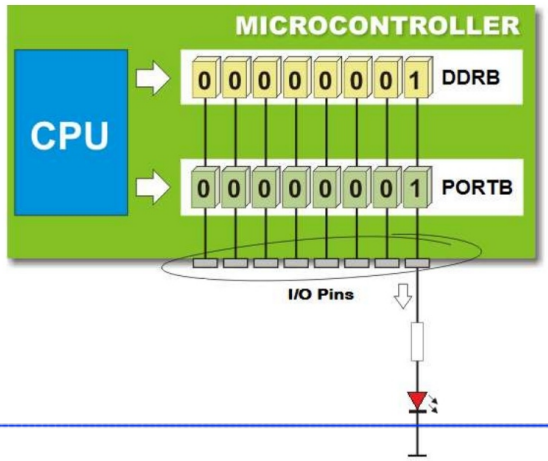
↳ Bits set to 0 indicate input pins.

∴ Pin 0 is configured to output.

Making an I/O Pin High

- Change the state of Pin 0 of Port B high
  - Use Set Bit in I/o register instruction

```
SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
```



SBI PortB, 0

↓

Port register bit 0 is set to 1.

∴ Voltage at the pin becomes 5V.

Delaying

- Engage CPU by running a loop and delay half a second

(1) Calculate clock cycle time → 16 MHz

$t = (1 / f) = (1 / (16 \times 10^6 Hz)) = 62.5 \times 10^{-9}$ Seconds

(2) Number of clock cycles required to achieve 0.5S delay

$n = (d / t) = (0.5 / (62.5 \times 10^{-9})) = 8 \times 10^6$ clock cycles

(3) Clock cycles consumed by one loop

→ bcs 8 bit word size

```
        LDI R18, 255     ; load 255 to R18
LOOP3:  DEC R18          ; decrement R18 by 1
        BRNE LOOP3       ; branch to LOOP3 if result is not 0
```

↳ outcome of prvs instruction

n_3 = (1 for DEC + 2 for branch) x (255 – 1) times +
(1 for DEC + 1 for fall through) x 1 time

$n_3 = (1 + 2) \times 254 + (1 + 1) \times 1 = 764$ clock cycles

(1) Cycle time $= \frac{1}{\text{frequency}} = \frac{1}{16 \times 10^6 Hz} = 62.5 \times 10^{-9}$ s

nano seconds.

(2) No $= \frac{\text{delay}}{\text{time for 1 clock cycle}} = \frac{0.5}{62.5 \times 10^{-9}} = 8 \times 10^6$ clock cycles

↓

If the CPU is engaged for this amt of clock cycles, can obtain 0.5s delay

LDI : Load Immediate.
(consumes 1 clock cycle)

DEC : Decrement
(1 clock cycle)

BRNE : Branch if not equal
(2 clock cycles)

Delaying with Nested Loops

- Clock cycles consumed by two nested loops

```
        LDI R17, 255
LOOP2:  LDI R18, 255
LOOP3:  DEC R18
        BRNE LOOP3      ] n3
        DEC R17
        BRNE LOOP2
```

Total Loops needed / Clock cycles consumed by n2 → No. of loops needed for n1

$\frac{8 \times 10^6}{195839} = 40.8 \simeq 41$

n_2 = (1 for LDI + n_3 + 1 for DEC + 2 for branch) x (255 – 1) times + (1 for LDI + n_3 + 1 for DEC + 1 for fall through) x 1 time

n_2 = (1 + 764 + 1 + 2) x 254 + (1 + 764 + 1 + 1) x 1

n_2 = 195839 clock cycles

larger / greater value

Delaying with Nested Loops

- Clock cycles consumed by three nested loops

```
        LDI R16, 41
LOOP1:  LDI R17, 255
LOOP2:  LDI R18, 255
LOOP3:  DEC R18
        BRNE LOOP3      ] n3   ] n2
        DEC R17
        BRNE LOOP2
        DEC R16
        BRNE LOOP1
```

n_1 = (1 for LDI + n_2 + 1 for DEC + 2 for branch) x (41 – 1) times + (1 for LDI + n_2 + 1 for DEC + 1 for fall through) x 1 time

n_1 = (1 + 195839 + 1 + 2) x 40 + (1 + 195839 + 1 + 1) x 1

n_1 = 8029562 clock cycles

Delay Calculation

- Delay achieved with nested loops

```
        LDI R16, 41
LOOP1:  LDI R17, 255
LOOP2:  LDI R18, 255
LOOP3:  DEC R18
        BRNE LOOP3
        DEC R17
        BRNE LOOP2
        DEC R16
        BRNE LOOP1
```

n_3 n_2 n_1

n = 1 for LDI + n_1
n = 1 + 8029562
n = 8029563 clock cycles
d = t x n (time x no. of clock cycles)
d = 62.5 x 10^{-9} x 8029563 = 0.501847625
d ≈ 0.5 Seconds

```
        LDI   R16, 72
LOOP 1: LDI   R17, 188
LOOP 2: LDI   R18, 196
        DEC R18
        BRNE LOOP3
        DEC R17
        BRNE LOOP2
        DEC R16
        BRNE LOOP1
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
        NOP
```

n = 7,999,992

→ Only missing 8 CCs to get exactly 0.5

∴ run 8 NOP to consume 1 cc per NOP wout doing anything to reach 8 mil.

n3 = 587

n2 = 111,107

n1 = 799,991

n = 799,992

Calling a Subroutine

- Execute the instructions for delaying half a second as a subroutine
  - Use CALL instruction

```
CALL DELAY              ; call DELAY subroutine
```

  - Use RETurn instruction to return from subroutine

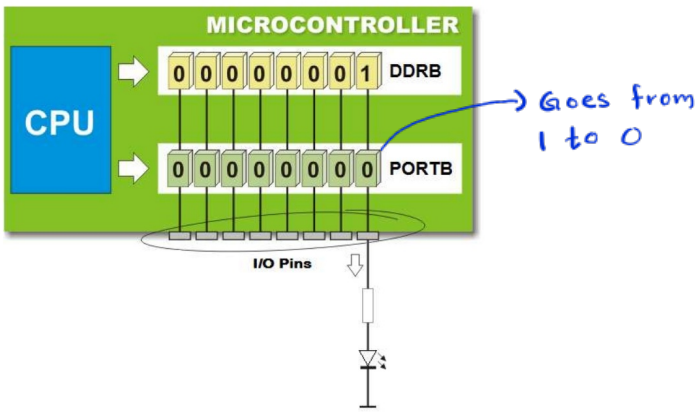
```
DELAY:  LDI R16, 41     ; load 41 to R16
LOOP1:  LDI R17, 255    ; load 255 to R17
LOOP2:  LDI R18, 255    ; load 255 to R18
LOOP3:  DEC R18         ; decrement R18 by 1
        BRNE LOOP3      ; branch to LOOP3 if result is not 0
        DEC R17         ; decrement R17 by 1
        BRNE LOOP2      ; branch to LOOP2 if result is not 0
        DEC R16         ; decrement R16 by 1
        BRNE LOOP1      ; branch to LOOP1 if result is not 0
        RET             ; return to calling point
```

↳ consume small subroutines & then return.

Making an I/O Pin Low

- Change the state of Pin 0 of Port B low
  - Use Clear Bit in I/o register instruction

```
CBI PORTB, 0    ; clear bit 0 of PORTB & turn off LED
```



Performing Tasks Indefinitely

- Perform the tasks such as change the state of Pin 0 of Port B high, delay 500ms, change the state of Pin 0 of Port B low and again delay 500ms, indefinitely
  - Use JuMP instruction

```
START:  SBI PORTB, 0    ; set bit 0 of PORTB & turn on LED
        CALL DELAY      ; call DELAY subroutine
        CBI PORTB, 0    ; clear bit 0 of PORTB & turn off LED
        CALL DELAY      ; call DELAY subroutine
        JMP START       ; jump to START
```

  - Infinite loop

Hardware error → Replace components

Software error → Debug the code.

*Bitwise Logical Operators.

→ set bit

PORTB = PORTB | 0000 0001

→ bitwise OR

xxxx xxxx | 0000 0001

= xxxx xxxx

using this only the interested value will be changed.

→ clear bit

PORTB = PORTB & 1111 1110

xxxx xxxx & 1111 1110

= xxxx xxx0

Arduino Code - 5 LED Blinker

```
int ledPin[] = {9,10,11,12,13};

void setup() {
    for (int i=0; i<5; i++) {
        pinMode (ledPin[i], OUTPUT)

void loop() {
    for (int i=0; i<5; i++) {
        digitalWrite (ledPin[i], HIGH);
        delay (500);
        digitalWrite (ledPin[i], LOW);
    }
}
```

C Code 5 LED Blinker.

```
# include <avr/io.h>
# define F_CPU 16000000UL
# include <util/delay.h>

int main() {
    arr[] = {1, 2, 3, 4, 5};
    for (int i = 0 , i<5, i++) {
        DDRB |= (1<<i);
    }

while (1)
{
    for (int i=0 ; i<5; i++)
    {
      PORTB |= (1 << arr[i]);
      _delay_ms(500);
      PORTB &= ~(1<< arr[i]);
    }
}
  return 0;
}
```

C Code 5 LED Blinker (Knight Rider)

```
#include <avr/io.h>
#define F_CPU 16000000UL
#include <util/delay.h>

int main(){
    arr[] = {1,2,3,4,5};
    for(int i=0; i<5; i++){
        DDRB |= (1<<i);
    }

while(1)
{
    for(int i=0; i<5; i++)
    {
      PORTB |= (1 << arr[i]);
      _delay_ms(500);
      PORTB &= ~(1<< arr[i]);
    }
}
  return 0;
}
```

Arduino Code - 5 LED Blinker (Knight Rider)

```
int ledPin[] = {9,10,11,12,13};

void setup() {
    for (int i=0; i<5; i++) {
        pinMode (ledPin[i], OUTPUT)

void loop() {
    for (int i=0; i<5; i++) {
        digitalWrite (ledPin[i], HIGH);
        delay (500);
        digitalWrite (ledPin[i], LOW);
    }
    for (int i=3; i>0; i--) {
        digitalWrite (ledPin[i], HIGH
```

04. Polling and Interrupt Handling

30 / 07 /2026

Polling and Interrupt

- Two approaches to preform more than one tasks seemingly at the same time
- Used to provide CPU's service to the peripheral devices when CPU is busy with another task

* How to perform a task while another task is in process. (Multiple tasks)

Polling

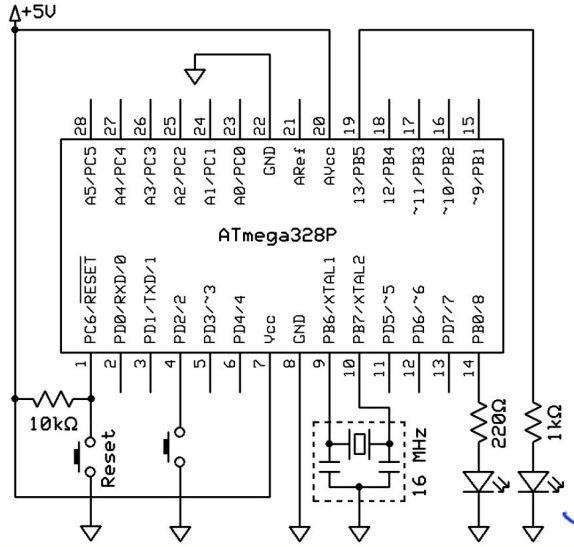
* Periodically check if the peripheral device to see if it requires any services. If required provide service, otherwise continue current task.

* CPU keeps on checking status of peripheral devices at regular intervals repeatedly while performing another task
- When the status condition of a device is met, CPU provides its service to the device and then resumes the task it was doing
- CPU then starts again monitoring the status of devices

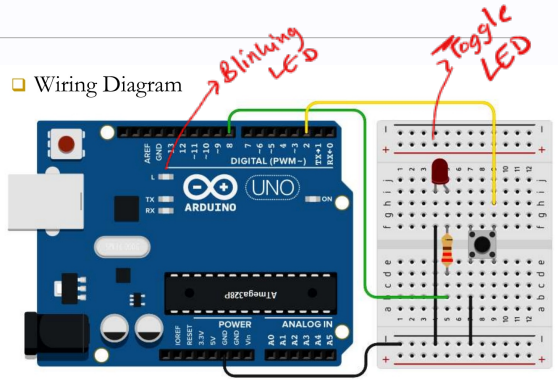
• System 5 : pollingBased Trigger (Toggles an LED on user input while blinking an LED in every 2 seconds using polling)

Hardware

- Schematic Diagram



- Wiring Diagram



* The pin connected ② must always be high. If there is a user input it'll be low.

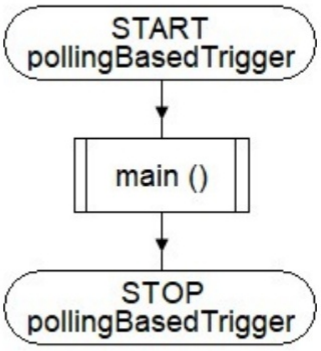
→ Blink every 2s

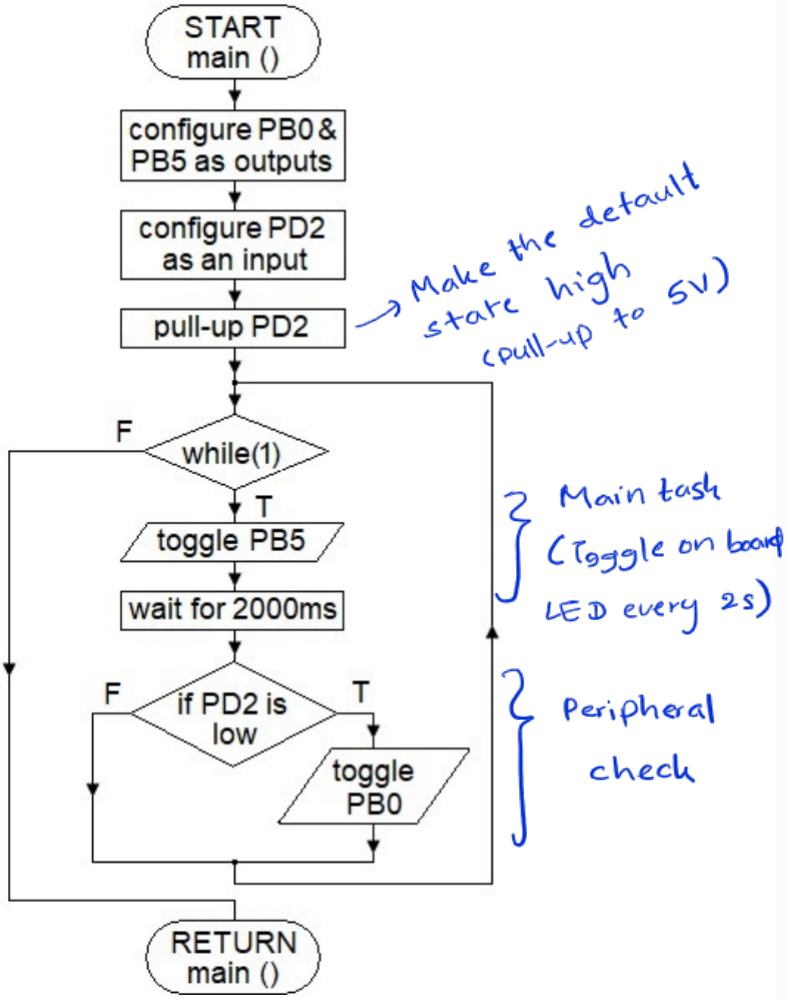
↓ user input

→ user input LED

Firmware

- Algorithm





C Program

```
1   // pollingBasedTriggerC.c
2
3   #include <avr/io.h>
4   #define F_CPU 16000000UL                // define CPU clock speed for delay.h
5   #include <util/delay.h>
6
7   int main(void)
8   {
9       DDRB = DDRB | 0b00100001;           // configure PB0 & PB5 as outputs
10      DDRD = DDRD & 0b11111011;           // configure PD2 as an input
11      PORTD = PORTD | 0b00000100;         // pull-up PD2
12      while (1)
13      {
14          PORTB = PORTB ^ 0b00100000;     // toggle state of PB5
15          _delay_ms(2000);                // delay 2000ms
16          if ((PIND & 0b00000100) == 0)   // if state of PD2 is low
17              PORTB = PORTB ^ 0b00000001; // toggle state of PB0
18      }
19      return 0;
20  }
```

/ /

if ((PIND & 00000100) == 0)

xxxx xxxx & 0000 0100

if (0000 0x00 == 0)

default 0000 0100 ==0

press button 0000 0000 == 0

PORTB = PORTB ∧ 0000 0001

xxxx xxxx ∧ 0000 0001

xxxx xxx$\overline{x}$

* If the input is given using a quick pulse the input will not be ditected. That is because for the input to be detected, the input should be received during the delay time that the cpu is checking the peripheral device. If the input actuator is a sensor, there might be multiple undected inputs due to this. This is the issue of polling

Polling

- Problems
  - If the state of a device changes too fast or the CPU's device monitoring interval is too long, the state change of the device may go undetected
  - No support for the concept of priority as CPU repeatedly performs all tasks in the same order

cannot priorotize if there are multiple devices

digitalRead → Gives the current state of the pin (high/low)

ex: digitalWrite (2, !digitalRead (2));

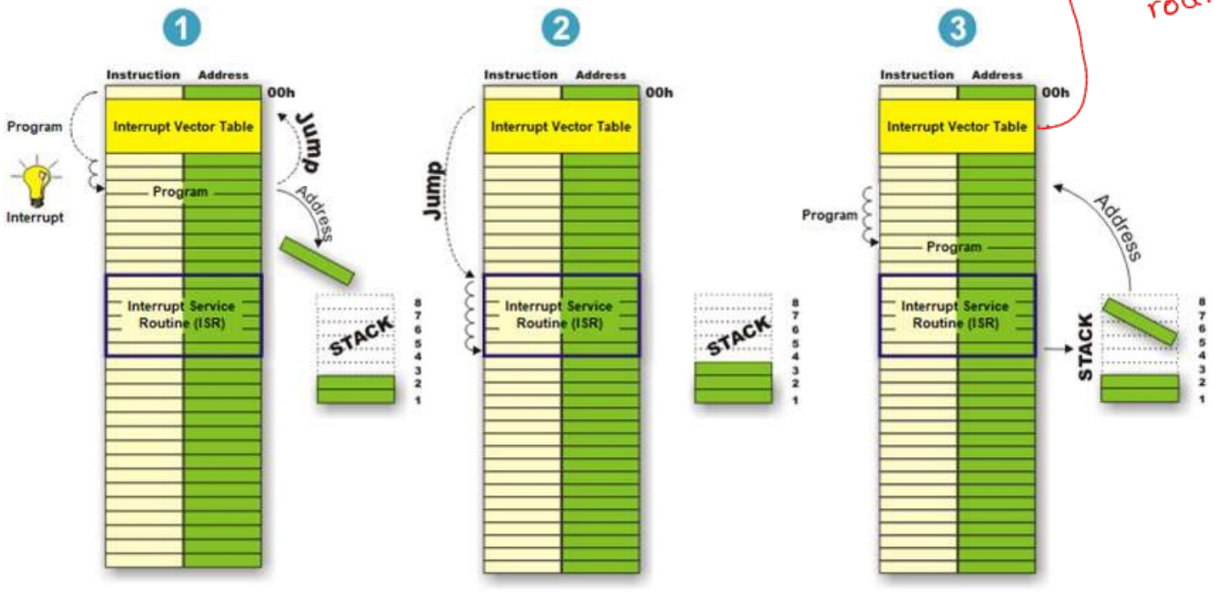
* Write the state of pin 2 to be the opposite of the current state of pin 2.

Interrupt

- Peripheral devices interrupt the CPU when they need CPU's service while CPU is busy with another task
- CPU suspends the currently performing task and provides its service to the interrupted device
- CPU then resumes the task it was doing
- Frees CPU from monitoring (polling) the state of peripheral devices
- Supports the concept of priority

Interrupt Handling

- Steps



* Every interrupt has an interrupt service routine (ISR)

Interrupt Vector Table

- Widely used Interrupt Vectors

Address	Interrupt	Interrupt_Vector
0x0002	External interrupt on INT0 pin	INT0_vect
0x0004	External interrupt on INT1 pin	INT1_vect
0x0006	Pin change interrupt on Port B pins	PCINT0_vect
0x0008	Pin change interrupt on Port C pins	PCINT1_vect
0x000A	Pin change interrupt on Port D pins	PCINT2_vect
0x0012	Timer/Counter 2 overflow interrupt	TIMER2_OVF_vect
0x001A	Timer/Counter 1 overflow interrupt	TIMER1_OVF_vect
0x0020	Timer/Counter 0 overflow interrupt	TIMER0_OVF_vect
0x0024	USART receive complete interrupt	USART_RX_vect
0x0028	USART transmit complete interrupt	USART_TX_vect
0x002A	ADC conversion complete interrupt	ADC_vect

Lower the addresses, higher the priority.

```
int flag = 0;
int main() {
    while(1) {
        if (flag == 0)
            // do something
        else
            // do something else
    }
    return 0;
}
```

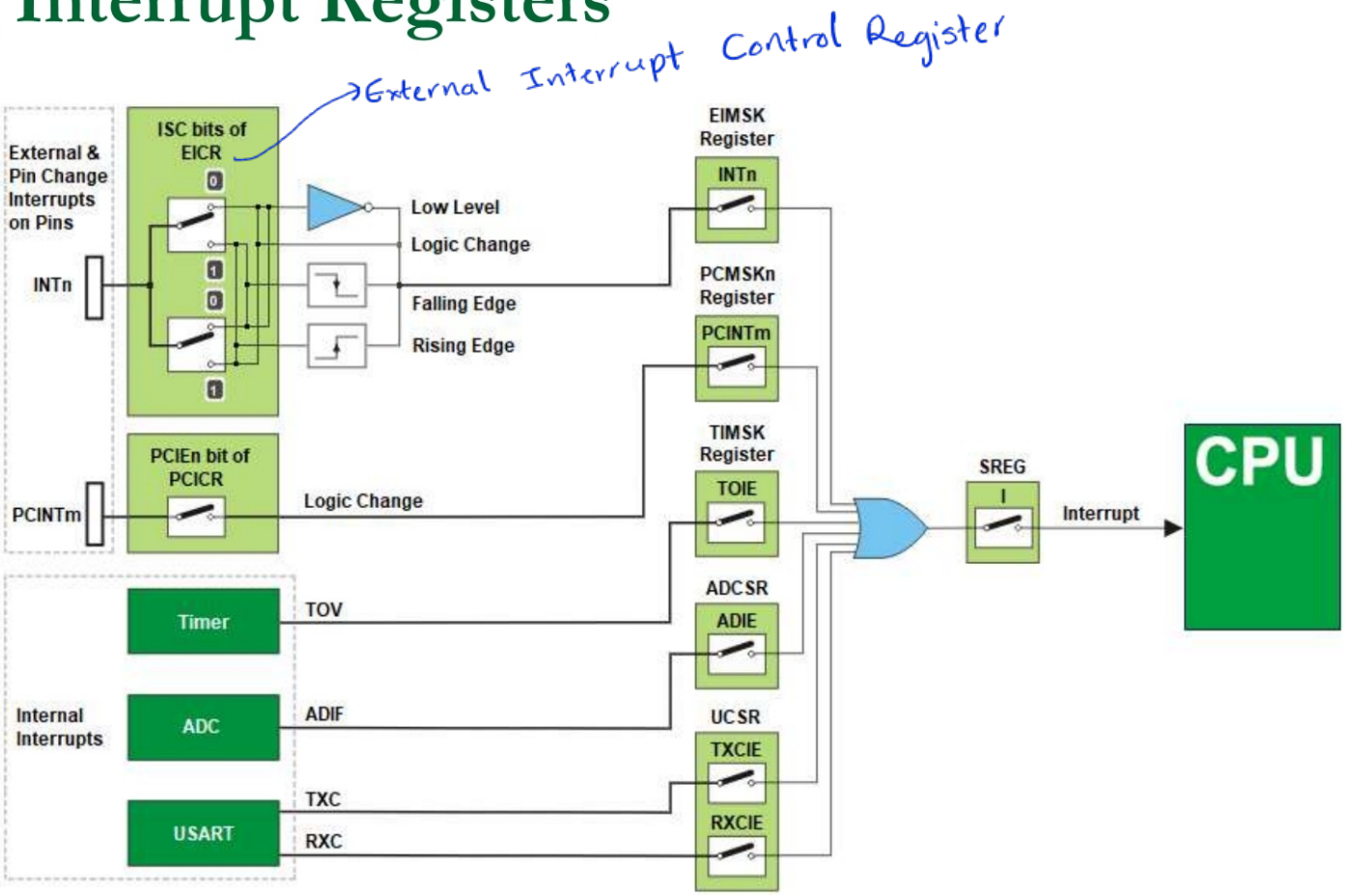
```
ISR(INT0_vect)
{
    flag = 1;
}
```

Set Interrupt → sei()

Clear Interrupt → cli()

} interrupt.h register

Interrupt Registers



External interrupt pins : INT Pins.

(1 class missed)

• Analog to Digital Conversions

ADC Reference Voltage

- ADC converts any analog input voltage between 0 and Vref to a number of discrete voltage intervals between 0 and $2^{\text{resolution}} - 1$
- Resolution is the number of bits used for ADC
- Total number of intervals = $2^{\text{resolution}}$
- Interval size = Vref / $2^{\text{resolution}}$
- analog voltage = number of intervals × interval size

$$Vin = ADCW \times \frac{Vref}{2^{resolution}}$$

ADCW → word

- Interval size is the smallest change in analog input voltage that ADC can distinguish

Vin → ADC → discrete no

↓ bet$\bar{w}$ 0 - Vref

↓ bet$\bar{w}$ 0 - ($2^{resolu\bar{n}}$ - 1)

*For our MC, operating voltage is 5V

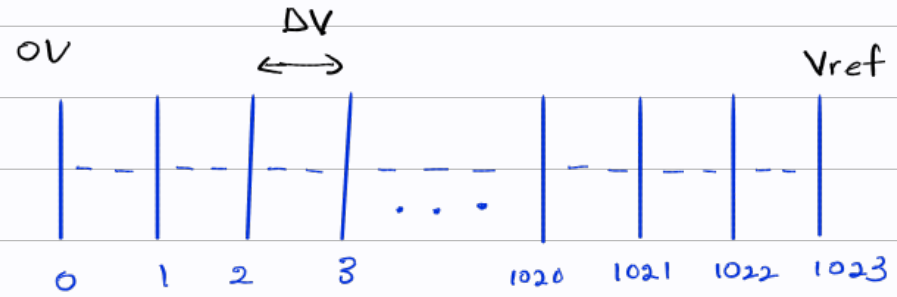
∴ default Vref : 5V

∴ resolu$^{\bar{n}}$: 10 bit

∴ For our mc : Vin → ADC → discrete no (n)

↓ 0 - 5V

↓ 0 - (2^{10} - 1) ie 0 - 1023



→ No. of points (Not gaps)

- Tot. No. of intervals = 2^{10} = 1024
- Interval size = $\frac{Vref - 0}{\text{Tot. intervals}} = \frac{5}{1024}$ → default voltage for Vref
- Vin = ΔV × n (discrete no)

/ /

ADC = 1	$0 \leq V_{in} < \Delta V$
ADC = 2	$\Delta V \leq V_{in} < 2\Delta V$
ADC = 1023	$1022\Delta V \leq V_{in} < 1023\Delta V$

* The smaller the range, the more accurate the voltage ADC gives. ∴ we must try to reduce the intervals (ΔV)

(1) Increase resolution

(2) If resolution is fixed, reduce Vref

Vin ⟶ [ADC] ⟶ ADCW

0-5V	Vref = 5V	(default)
0-1V	Vref = 1.1V	(internal)
0-2.5V	Vref = 2.5V	(externally)

ADC Registers

- ADMUX (ADc MUltipleXer selection) register



  - REFS1 (REFerence Selection 1) and REFS0 bits set Vref
  - Used to specify Vref and thereby interval size of ADC
  - In Arduino, use `analogReference(type)` ⟶ Call in the setup once is enough.

REFS1	REFS0	Vref	type
0	0	Set to V_{AREF} supplied externally	EXTERNAL
0	1	Set to AVcc internally	DEFAULT
1	0	Reserved	
1	1	Set to 1.1V internally	INTERNAL

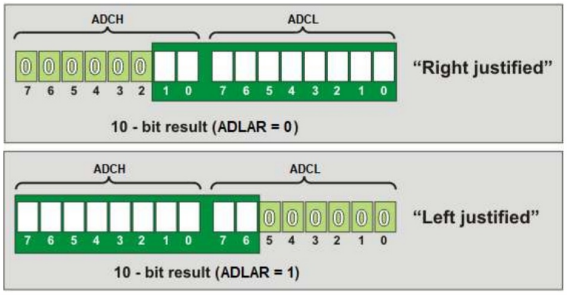
Analog to Digital Conversion_050229f163aaf5f33ecee2079231bf66.pdf

ADC Registers

- ADMUX register

ADMUX	REFS1	REFS0	ADLAR	–	n/a	MUX2	MUX1	MUX0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - ADLAR (ADc Left Adjust Result) bit specifies whether the conversion result is left justified



ADC Registers

- ADMUX register

ADMUX	REFS1	REFS0	ADLAR	–	n/a	MUX2	MUX1	MUX0	Bit name
	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	

  - MUX2 (MUltipleXer selection 2), MUX1 and MUX0 bits are used to multiplex analog input channels

MUX2	MUX1	MUX0	Channel
0	0	0	ADC0
0	0	1	ADC1
0	1	0	ADC2
0	1	1	ADC3
1	0	0	ADC4
1	0	1	ADC5

C Program

```c
// voltMeterC.c
#include <avr/io.h>                          // include I/O definitions
int main(void)
{
    DDRB = DDRB | 0b00011111;                // configure PB0 to PB4 as outputs
    DDRC = DDRC & 0b11111110;                // configure PC0 (ADC0) as an input
    ADMUX = (ADMUX & 0b01111111) | 0b01000000; // set Vref to AVcc (5V)
    ADMUX = ADMUX & 0b11111000;              // multiplex PC0 as analog input channel (ADC0)
    ADCSRA = ADCSRA | 0b00000111;            // set ADC clock speed to F_CPU/128
    ADCSRA = ADCSRA | 0b10000000;            // enable ADC
    while (1)
    {
        ADCSRA = ADCSRA | 0b01000000;        // start ADC
        while(ADCSRA & 0b01000000);          // poll till ADC is over
        float vIn = ADCW * 5.0 / 1024;       // compute analog input voltage
        PORTB = PORTB & 0b11100000;          // set PB0 to PB4 low & turn off LEDs
        if (vIn > 4)
           PORTB = PORTB | 0b00011111;       // if voltage > 4V, set PB0 to PB4 high & turn on 5 LEDs
        else if (vIn > 3)
           PORTB = PORTB | 0b00001111;       // if voltage > 3V, set PB0 to PB3 high & turn on 4 LEDs
        else if (vIn > 2)
           PORTB = PORTB | 0b00000111;       // if voltage > 2V, set PB0 to PB2 high & turn on 3 LEDs
        else if (vIn > 1)
           PORTB = PORTB | 0b00000011;       // if voltage > 1V, set PB0 to PB1 high & turn on 2 LEDs
        else if (vIn > 0)
           PORTB = PORTB | 0b00000001;       // if voltage > 0V, set PB0 & turn on 1 LED
    }
    return 0;
}
```

→ At least one must be floating point

TRANSDUCERS

Transducer

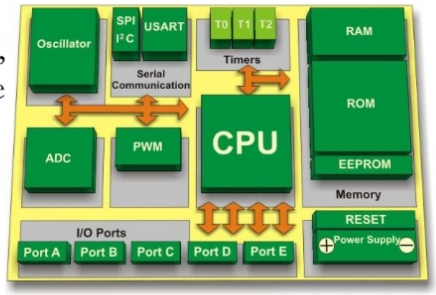
- Converts energy from one form to another
- Used at the boundary of an embedded system
- Categorized by the direction of conversation
  - Sensors
  - Actuators

PIR - Pyro-electric InfraRed

Timing and Counting

Timer

- Increments on each clock pulse when oscillator is connected
- As frequency is known, using pulse count, time elapsed can be determined
- Operates as a Counter, when an external pulse source is connected
  - Used to count events
- What's the need?



↳ The same thing can be done using software. So why do we need this?

* Timer is connected to a switch and timer is reset each time the button is pressed. Then the timer acts as a counter.

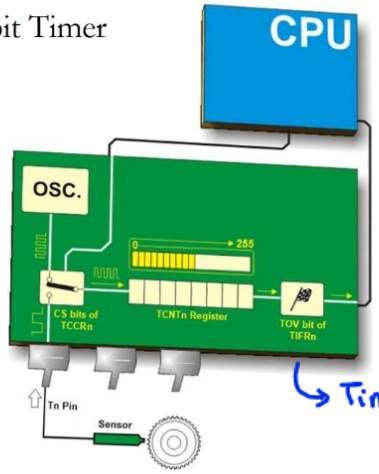
↗ (generate pulses)

* When an oscillator is connected to the timer, we can find elapsed time.

→ If software is used for timing & counting, it ties up the CPU. In MCs we have a very small CPU, so this is a huge burden. ∴ we can leave CPU for computational works and use timers (which is an external peripheral device) for this purpose.

Timing and Counting

- How an 8-bit Timer Works?



↳ Timer Overflow Bit

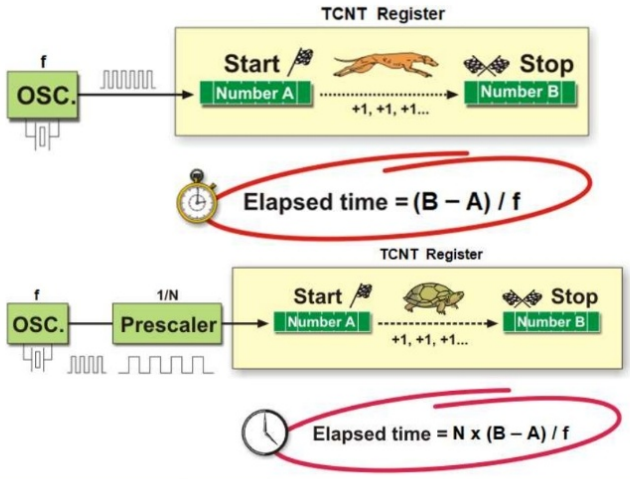
* Oscillator is connected to Timer. At each clock pulse timer increments.
* Increments happen 0-255. Then overflows and goes back to 0.
* TRFI - Timer Interrupt Flag Register.

↓

when Timer Overflow's TOV bit is set.

This tells CPU that time overflowed.

Determining Elapsed Time



TOV set it to clear

W1C : Write One to Clear